

# Kenny — Technical Architecture

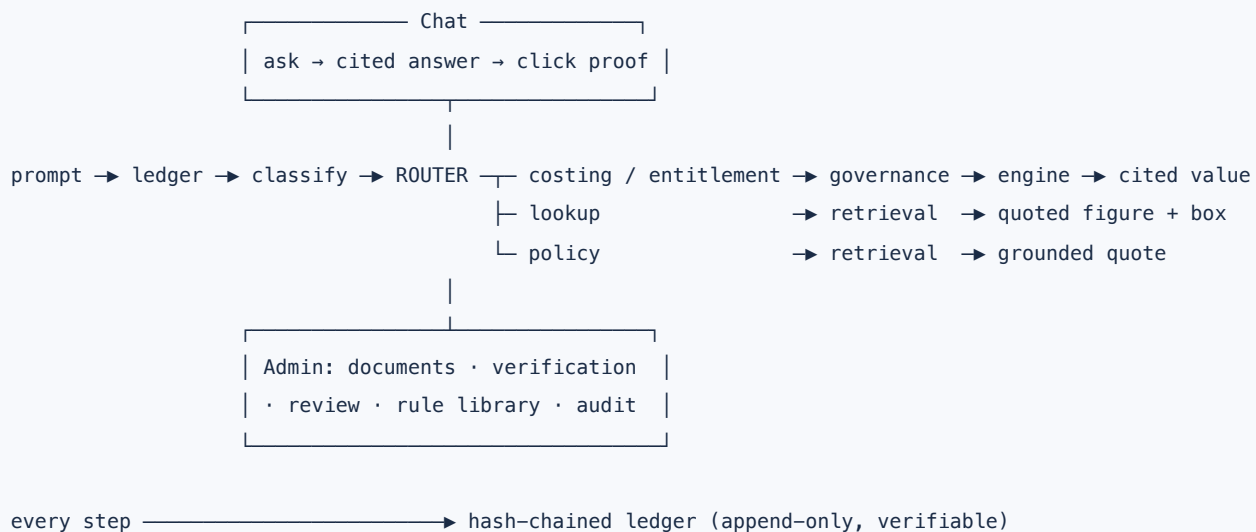
## Engineering companion to the PRD

The PRD says what Kenny is and why. This says how it is built: the structures, the contracts between them, and the invariants a change must preserve. Where they overlap, the PRD owns intent and this owns behaviour.

**Status:** ✅ built · 💠 designed, not built · ⌚ scale roadmap.

## 1. System overview

One application, two surfaces, one spine. The architectural bet: **the AI is a translation layer, never a decision layer**. It reads language into structured data; deterministic code selects the governing contract, chooses the rules, and does every arithmetic step. Correctness is therefore a property of the deterministic layer plus a human gate — not of the model.



## 1.1 Components

| Module                      | Responsibility                                                                                                                                                                     | Why it is its own thing                                                          |
|-----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| <code>app.py</code>         | HTTP routes, intent router, orchestration                                                                                                                                          | one spine, handlers behind it                                                    |
| <code>llm.py</code>         | <b>the only AI surface</b> — classify, parse, draft, tag, rank, compose                                                                                                            | quarantine the model to one auditable file                                       |
| <code>engine.py</code>      | deterministic interpreter: differentials → base → premiums → rounding                                                                                                              | the money path; imports no AI                                                    |
| <code>rulesl.py</code>      | rule schema, sandboxed expression evaluation, validation                                                                                                                           | rules are untrusted data by default                                              |
| <code>governance.py</code>  | (unit, date) → governing document(s); supersession                                                                                                                                 | law selection is a fact, not a search                                            |
| <code>ingest.py</code>      | document → clauses + bounding boxes; title extraction; chunking                                                                                                                    | provenance is the product                                                        |
| <code>index.py</code>       | <code>SearchBackend</code> : local hybrid (BM25 ⊕ embeddings), scale backend behind the same interface                                                                             | swap retrieval without touching callers                                          |
| <code>retriever.py</code>   | catalog-level document routing (a fallback to governance)                                                                                                                          | —                                                                                |
| <code>catalog.py</code>     | per-document index: title, department, tags, summary, clauses                                                                                                                      | candidate identification                                                         |
| <code>caseio.py</code>      | case bundle loader; the rule vocabulary ( <code>known_facts</code> , values, scenarios)                                                                                            | one source of truth for what a rule may reference                                |
| <code>dataadapter.py</code> | <code>DataSource</code> : CSV today, DB/API behind the same interface                                                                                                              | classification-table source is swappable                                         |
| <code>ledger.py</code>      | hash-chained append-only event log + <code>verify()</code> ; events HMAC-signed under <code>KENNY_LEDGER_KEY</code> (kept off-volume) with the head hash anchored to platform logs | tamper-evident audit — rewrite needs the key, truncation fails the anchored head |
| <code>audit.py</code>       | per-query trail assembly; frozen answer snapshots                                                                                                                                  | an answer stays reproducible after the contract renews                           |
| <code>pdfview.py</code>     | render a page and overlay the citation box                                                                                                                                         | citations you can see                                                            |
| <code>auth.py</code>        | two-role auth, per-IP rate limit, refuse-to-start-misconfigured                                                                                                                    | a shared link must not hand out the approval gate                                |

Everything jurisdiction-specific lives in `cases/<name>/` ; `core/` is case-agnostic.

## 1.2 The seams that keep the roadmap cheap

Each is an interface with a working implementation and a documented alternative: `SearchBackend` (local ↔ cluster), `DataSource` (CSV ↔ HRIS), and **the AI boundary itself** — every `llm.*` function has a deterministic fallback, which makes the test suite hermetic and proves the model is not load-bearing.

## 2. The AI boundary

Every AI call is one of seven functions in `llm.py`. None computes money; none decides which contract or rule applies. They exchange **structured JSON** with the deterministic core.

| Function                        | When             | Emits                                                                    | Fallback           |
|---------------------------------|------------------|--------------------------------------------------------------------------|--------------------|
| <code>classify_intent</code>    | per question     | costing   lookup   entitlement   policy                                  | keyword router     |
| <code>extract_department</code> | per question     | which unit's contract                                                    | cue match          |
| <code>parse_intent</code>       | per question     | <code>{subjects, hours, date}</code> , resolved to classification labels | regex + name match |
| <code>answer_policy</code>      | per question     | prose composed <b>only from retrieved clauses</b>                        | quote top passage  |
| <code>rank_documents</code>     | routing fallback | ranked candidate documents                                               | catalog scan       |
| <code>tag_document</code>       | per ingest       | department, tags, summary                                                | heuristic tags     |
| <code>draft_rules</code>        | per scenario     | proposed rules + data-gap report                                         | offline stub       |

**Observability.** Every call records itself to a per-request trail ( `llm.record()` , a `contextvars` context so concurrent requests don't cross) that the app drains into the ledger as `llm.call` events: which function, `claude | fallback | error` , model, latency. This answers a question the rest of the trail cannot — *did a model run at all?* — because with a fallback behind every call, an absent model is invisible unless recorded. The chat drawer surfaces it: an answer states whether AI was used and where, so a reader can confirm no figure came from a model.

## 3. Domain model & governance

- **Subjects** are **classifications**, never named people: an attribute bag ( `bargaining_unit` , rank, step, shift, certifications, base rate) keyed by a classification label (PRD §6a). The engine never depends on identity — a subject is the set of facts a rule keys on. Every computed amount is **per member** of the classification; multiplying by a staffing count is deliberately left to the reader (headcount churns weekly and lives in the HR system). Payroll reconciliation against real individuals is a separate audit mode, not the costing path.
- **Documents** declare `doc_type` ( `MOU | amendment | salary-schedule | policy` ), `bargaining_unit` , and `effective_start/end` .
- **Rules** belong to a document version; ids are namespaced `doc_id:rule_id` .

```
subject (classification):
{ "classification": "Firefighter/Paramedic (56 hr, top step)",
  "bargaining_unit": "firefighters-local-3535", "department": "fire",
  "rank": "Firefighter/Paramedic", "base_hourly": 53.40, "shift": "56-hour" }
```

The table that feeds this is `class × step × shift × certs × rate` — a position table HR already maintains, ingested through the `DataSource` interface. A question resolves to classifications **deterministically, by attribute filter**: "*the graveyard police classifications*" is `shift = Graveyard AND department = police` over the table (exact label mentions

win outright; described attributes constrain AND-across-fields, OR-within-a-field). The model may propose labels, but only labels that exist in the table survive normalization.

"Which rules govern this subject on this date" is a **deterministic lookup**, not a search:

```
governance.resolve(units, date_iso, sources) -> GovResult(doc_ids, resolved, reason)
governance.apply_supersession(rules, sources, doc_ids) -> (kept, dropped)
```

Governance resolves two orthogonal axes. **Date** decides which document *versions* apply (a salary schedule effective this year, a side letter effective mid-term). **Specificity** decides which of two applicable rules *wins*: a `scope_rank` derived from `doc_type` — `statute < policy < MOU < amendment` — so the more specific document overrides the more general (lex specialis). *Everyone is off on the holiday, except police assigned to a shift* resolves because the police MOU outranks the citywide policy — no tuned number, a fact about document authority.

**Supersession**: an amendment declares what it replaces ( `supersedes: {doc_id, clauses}` ); the superseded base rules are dropped before the engine runs, but only when a live replacement exists — an un-approved amendment must not silently delete the rule it was meant to update.

## 4. The rule model

A rule is **data interpreted by one generic engine** — never code. New pay math is authored and verified, not programmed.

```
{ "id": "doc:graveyard_differential",
  "role": "differential",           // base | differential | premium | exception ($4.1)
  "result_type": "currency",       // currency | days | hours | date | boolean
  "pay_basis": "hourly",           // hourly | per_shift | per_pay_period | monthly | annual | one_time
  "when": "subject_shift == 'Graveyard'",
  "set": {"effective_base": "effective_base * 1.055"}, // differential adjusts a fact
  "compute": "effective_base * 2.5 * hours",           // base/premium yields an amount
  "citation": {"doc_id": "...", "clause": "6.1", "page": 8, "bbox": [x0,y0,x1,y1]},
  "status": "ratified", "approver": "analyst" }
```

- **Expressions are sandboxed strings** ( `simpleeval` ) over declared facts — data, not executable code.
- **result\_type** says what the number *means*; the engine is unit-agnostic, so one machine returns `$4,430.40` or `5 days`. Only rules of the question's type compete.
- **status defaults to proposed**. Only `ratified` + a real approver loads into the engine.
- **scope\_rank and precedence are derived, not authored**. The drafter never emits a priority number; it cannot see the other documents, so it cannot rank against them.

### 4.1 Composition — pay is a stack, not a contest

The engine composes rules by **role** rather than picking one winner, because compensation is additive and layered:

| role         | mechanic                                  | how many fire                      | ordered by                       |
|--------------|-------------------------------------------|------------------------------------|----------------------------------|
| differential | set — adjusts <code>effective_base</code> | all matching                       | authored order (they accumulate) |
| base         | compute — the pay formula                 | <b>exactly one</b> (most specific) | scope, then priority             |
| premium      | compute — an added amount                 | all matching, <b>summed</b>        | authored order                   |
| exception    | suppress / cap another term               | ◆ designed                         | —                                |

```
line_total = base(chosen) + Σ premiums
```

after differentials adjust the facts  
(per MEMBER of the classification – no  
headcount multiply; that is the reader's)

Roles are what stop a \$250 stipend or a court-time rule from *winning* an eight-hour holiday shift: a premium is not a base candidate, it adds. A percentage *of the base rate* is a differential (it compounds through later multipliers); a flat add-on is a premium.

## 4.2 Pay basis — the right unit for the question

A single-shift cost includes only `hourly` and `per_shift` terms. An annual uniform allowance, a monthly medical contribution, a year of life insurance are real money in the **wrong unit** for "*what does this shift cost?*" — the engine filters them out by `pay_basis` rather than adding a year of benefits to one shift. A different question (annual compensation) passes a different basis scope.

## 4.3 Attributes — how a generic rule reaches a specific person

A rule keys on facts ( `subject_shift` , `subject_bilingual` , `event` ). A fact is **looked up** (the classification table), **stated** (the question), or **asked for** — never invented. A clause that pays for something the data cannot identify (a detective-bureau premium with no assignment field, court time with no event fact) is **not drafted** — it is reported as a data gap naming the missing field. Broadening it to fire on everyone is the one thing the drafter must never do.

# 5. Flows

## 5.1 Ingest — extract and index, draft nothing

```
document -docling→ clauses + page + bbox → chunk → SearchBackend.index()
      |→ extract_title (from the cover, not a filename)
      |→ tag_document (department, tags, summary)
      |→ catalog.upsert
```

Ingest makes a document **searchable** — policy Q&A and lookup work immediately — and drafts **zero** rules. Rules are authored per scenario (§5.2). This is deliberate: modeling a whole contract up front produces hundreds of unverified rules; modeling per known-answer produces a handful, each provable.

Parsing degrades in tiers so a document is never silently empty: layout-aware extraction with real bounding boxes; a fallback for table-only pages the layout model misreads as images; and a raw-text tier (page-level citations) for scans. A

declared document with no file is a hard error, not an empty catalog entry.

## 5.2 Author rules for a scenario

A **scenario** is a known answer — a real paystub, or an analyst's hand-computed figure — carrying the people, the inputs, the expected amount, and a `branch_query` naming the pay it exercises.

```
scenario → governance.resolve → governing doc(s)
    → retrieve only the clauses the branch_query matches (a handful, not the whole contract)
    → exclude clauses that already have a live rule (draft only the gap)
    → draft_rules over just those clauses
    → verify the drafts reproduce the known amount
    → queue the few rules for human review, tagged to the scenario
```

Because the drafting task is small and focused, the model gets roles, units, and compounding right where a whole-contract task is fragile. Clauses that surface but aren't the scenario's type, or that the drafter refuses (event pay with no fact), don't become rules — they are answered by policy Q&A or reported as data gaps.

## 5.3 Answer a question at run time

```
prompt → classify_intent
├ policy / lookup → retrieve within the governing docs → answer_policy (from retrieved text only)
└ costing / entitlement → parse_intent → classification join → governance.resolve
    → load RATIFIED rules —(none)→ refuse, with a reason
    → engine.calculate → enrich citations → snapshot → cited answer
```

`parse_intent` normalizes loose values to the data's exact vocabulary ( "Saturday" → `Sat` ) and resolves class descriptions to classification labels by deterministic attribute filter (§3); otherwise exact matching fails silently. A costing question with no approved rule **refuses** — it never guesses a number.

## 5.4 Retrieval — a three-layer funnel

Each layer is cheaper and more certain than the next.

|   |                            |                                                                                                                                                      |
|---|----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | GOVERNANCE (deterministic) | who + date → the ONE governing document version                                                                                                      |
| 2 | METADATA SHORTLIST         | department → that unit's docs + citywide policies                                                                                                    |
| 3 | HYBRID SEARCH              | BM25 (exact tokens: \$, \$, a classification) ⊗ embeddings (paraphrase)<br>fused by reciprocal-rank fusion → clauses with doc · clause · page · bbox |

Governance is a fact and the only layer allowed to decide a costing answer. Semantic search is the last and weakest layer, ranking clauses *inside* a set the deterministic layers already narrowed. Pure semantic search is wrong for contracts — answers hinge on exact tokens ( `$9.2` , `$150` , a classification name) — so retrieval is **hybrid by requirement**, and there is no vector database until corpus size demands one.

## 5.5 Audit

Every action appends a `{seq, ts, actor, type, payload, prev_hash, hash}` event; `verify()` walks the chain and detects any edit or deletion. Answer **snapshots** freeze the rule versions used, so a past answer stays reproducible after the

contract renews. Export hands the verified chain to council, union, or legal.

## 6. Verification & the approval gate

Verification is the heart of the trust model. It has three states, and `fail` means exactly one thing.

| status               | meaning                                                                                              | blocks approval? |
|----------------------|------------------------------------------------------------------------------------------------------|------------------|
| <code>pass</code>    | the rules reproduce the known answer                                                                 | —                |
| <code>fail</code>    | <b>authored</b> rules disagree with the known answer                                                 | <b>yes</b>       |
| <code>pending</code> | not fully authored: no rule covers the scenario yet, <i>or</i> a governing document contributes none | no               |

The distinction is load-bearing. A number computed from an *incomplete* library (an amendment whose replacement rule nobody drafted yet) is *not-yet-authored*, not *wrong* — treating it as failure would train reviewers to ignore red. Only authored-rules-disagree blocks.

**Approval is a regression guard** (`admin_ratify`), not a completeness demand. It compares scenario status before and after the approval, over the live library **merged by id** with the selection: the scenario the rules were drafted for must pass, a scenario that was passing must not drop to fail, and uncovered scenarios stay pending. This makes incremental, scenario-by-scenario authoring work where a "every scenario must pass on every approval" gate would deadlock. A **release gate** (`prepare_deploy`) is stricter — it refuses to build a deployable image whose known answers do not all pass, so a half-authored state cannot ship as a product.

**At scale**, a scenario is not a single paystub but the payroll export — every paycheck, per person. Verification becomes per-paycheck reconciliation across the real input distribution; coverage is the fraction of real payroll reproduced. The engine already computes per-person amounts, so this is an extension of the scenario format (expected *per person*, not one total), not a redesign. ◆

## 7. Error handling

The governing principle: **degrade loudly, refuse rather than guess, never let a failure look like a success**. The categories are handled deliberately differently:

| Category               | Example                                                  | Handling                                                |
|------------------------|----------------------------------------------------------|---------------------------------------------------------|
| Missing input          | a declared document with no file                         | <b>hard error</b> , named                               |
| Degraded input         | a scan with no text layer                                | fall a tier, warn, page-level citation                  |
| Insufficient authority | no approved rule for a cost                              | <b>refuse with a reason and a next action</b>           |
| Unresolvable           | no date; evidence spans units                            | <b>ask</b> — never pick                                 |
| Unsatisfiable          | pay for an event no classification attribute can confirm | refuse and explain — it appears under Declined-to-model |
| AI unavailable         | no key, or a model error                                 | deterministic fallback, <b>recorded</b> as such         |

A silent fallback is the most dangerous handler in the system — an `except: pass` around a model call is indistinguishable from success. Every fallback records that it fired; "no exception" is not evidence of a result.

## 8. Storage & scale

**Today: files, no database.** A case is a directory: PDFs, extracted `catalog.json`, the search index, rule libraries (proposed + ratified), and the `ledger.jsonl`. This is a decision, not debt — **rules in version control are diffable**, so "who changed this differential, when, against which clause" is answered by the history itself, which a rules table gives up.

Each store has a different scale destination, reached at a different trigger:

| Store        | Today                     | At scale                                    | Trigger                                         |
|--------------|---------------------------|---------------------------------------------|-------------------------------------------------|
| Documents    | disk / image              | object storage, signed URLs                 | more than one machine                           |
| Extraction   | <code>catalog.json</code> | relational DB (document → version → clause) | re-ingesting to fix one field stops being cheap |
| Search index | JSONL                     | cluster backend (same interface)            | ~10 <sup>4</sup> chunks                         |
| Rules        | JSON in git               | <b>JSON in git</b>                          | never — the history is the point                |
| Ledger       | <code>ledger.jsonl</code> | append-only DB                              | concurrent writers on more than one machine     |

The **ledger is the one store with a genuine database-shaped problem** and the only one whose loss is unrecoverable — documents re-fetch, the index re-derives, but the record of who decided what cannot be reconstructed. It is the first migration when a jurisdiction outgrows a single machine.

## 9. Security & deployment

A local run and a shared link are different products. On any shared deploy:

- **Two-role auth, enforced per-path.** *Asking* and *deciding what the contract means* are different jobs; the viewer credential reaches Chat and the documents it cites, and only the admin credential reaches `/admin/*` (ratify, upload,



ingest, ledger). The app **refuses to start** misconfigured — including a missing ledger HMAC key — so a silent security failure becomes a loud startup failure.

- **Per-client rate limit** on the chat endpoint (real client IP via `Fly-Client-IP` on Fly) — one loop must not drain the model budget — plus exponential backoff on failed logins and an Origin check that refuses cross-site state-changing requests (CSRF).
- **Source integrity.** Every ingested PDF's SHA-256 is recorded; serving, costing, and ratified citations verify against it, and a re-ingest that moves or changes cited evidence marks the dependent rules stale until a human re-verifies them.
- **Path-safe document serving.** A `doc_id` from a URL is resolved through the manifest and confined to the case directory; it is never joined into a filesystem path.
- **Sandboxed rule expressions.** Rules are authored by a model and never executed as code.
- **Secrets stay out of the image.** Set out of band, never baked into a layer.
- **Single writer.** The ledger's file lock and the in-process job registry assume one writer; the deploy pins one machine until the ledger moves to a database.

The corpus, models, and approved rules are baked at build time; the ledger lives on a persistent volume so approvals and the audit trail survive redeloys. **Basic auth is not identity** — it separates roles but cannot attribute an approval to a named person; SSO + per-user attribution is the gap between a demo and a system a jurisdiction runs on, and *who approved this rule* is a question a union asks.

---

## 10. Onboarding checklist (any jurisdiction)

1. **Declare each document's governance** — unit and effective dates. The one thing no machine extracts. Verify two people on dates spanning an amendment resolve to different documents.
2. **Ingest.** Assert every document parsed with real citations, and clause counts are plausible. Policy Q&A works now.
3. **Map the classification table** to the baseline attribute vocabulary; case-sensitive values must match the data exactly. No employee names — classifications only (PRD §6a).
4. **Provide known answers** (paystubs). Per scenario, draft → review → approve, watching that approving one unit's rules never breaks another's passing scenario.
5. **Ask**, and confirm an unsatisfiable question refuses rather than guesses.

**The deployed corpus is real:** the Central Fire District of Santa Cruz County's published MOUs (scans → OCR → docling, ~1,860 clauses) and master salary schedule. Its two verification scenarios are analyst-derived from the documents themselves (1.5× overtime × the schedule's real rate = \$640.80; three bereavement shifts per Article XIV) — the PRD's explicit fallback until the district supplies real payroll. The guided test path for reviewers is PRD §12. Nothing in `core/` is case-specific.